

Government Engineering College Kishanganj

Department of Computer Science and Engineering

COMPUTER NETWORKS LAB-04

TCP Client-Server Student Information Retrieval

Objective

To implement a TCP-based client-server application in C that performs query-based retrieval of student information.

Problem Statement

Design and implement a TCP client-server program in C. The server maintains student records in a text file. The client sends a Registration Number and the server returns the corresponding student details. The connection remains active for multiple queries and terminates when the client sends **bye**.

Sample students.txt

```
1001 Amit ECE GEC_Kishanganj
1002 Rahul CSE GEC_Kishanganj
1003 Rakesh ME GEC_Kishanganj
1004 Priya CSE GEC_Kishanganj
1005 Neha EEE GEC_Kishanganj
```

Server Program — server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

void find_student(char *reg_no, char *response)
{
    FILE *fp = fopen("students.txt", "r");

    if (fp == NULL) {
        strcpy(response, "Error opening file");
        return;
    }

    char file_reg[50], name[50], branch[50], college[100];
    int found = 0;

    while (fscanf(fp, "%s %s %s %s",
                  file_reg, name, branch, college) != EOF)
    {
        if (strcmp(file_reg, reg_no) == 0)
        {
            sprintf(response,
                    "Name: %s, Branch: %s, College: %s",
                    name, branch, college);
            found = 1;
            break;
        }
    }

    if (!found)
        strcpy(response, "Data not found");

    fclose(fp);
}

int main()
{
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE], response[BUFFER_SIZE];
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

```

if (server_fd < 0) {
    perror("Socket failed");
    exit(EXIT_FAILURE);
}

address.sin_family = AF_INET;
address.sin_addr.s_addr = INADDR_ANY;
address.sin_port = htons(PORT);

if (bind(server_fd, (struct sockaddr *)&address,
        sizeof(address)) < 0) {
    perror("Bind failed");
    close(server_fd);
    exit(EXIT_FAILURE);
}

if (listen(server_fd, 3) < 0) {
    perror("Listen failed");
    close(server_fd);
    exit(EXIT_FAILURE);
}

printf("Server is listening on port %d...\n", PORT);

new_socket = accept(server_fd,
                    (struct sockaddr *)&address,
                    (socklen_t *)&addrlen);

if (new_socket < 0) {
    perror("Accept failed");
    close(server_fd);
    exit(EXIT_FAILURE);
}

printf("Client connected.\n");

while (1)
{
    memset(buffer, 0, BUFFER_SIZE);
    memset(response, 0, BUFFER_SIZE);

    int bytes = read(new_socket, buffer, BUFFER_SIZE - 1);

    if (bytes <= 0)
        break;

    buffer[bytes] = '\0';

    if (strcmp(buffer, "bye") == 0)
        break;

    find_student(buffer, response);
    send(new_socket, response, strlen(response), 0);
}

close(new_socket);
close(server_fd);

printf("Connection terminated.\n");
return 0;
}

```

Client Program — client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main()
{
    int sock = 0;
    struct sockaddr_in serv_addr;
    char reg_no[BUFFER_SIZE], buffer[BUFFER_SIZE];

    sock = socket(AF_INET, SOCK_STREAM, 0);

    if (sock < 0) {
        perror("Socket creation failed");
        return 1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    if (inet_pton(AF_INET, "127.0.0.1",
                  &serv_addr.sin_addr) <= 0) {
        perror("Invalid address");
        return 1;
    }

    if (connect(sock, (struct sockaddr *)&serv_addr,
                sizeof(serv_addr)) < 0) {
        perror("Connection failed");
        return 1;
    }

    printf("Connected to server.\n");

    while (1)
    {
        printf("\nEnter Registration Number (or type 'bye' to exit): ");

        fgets(reg_no, BUFFER_SIZE, stdin);
        reg_no[strcspn(reg_no, "\n")] = '\0';

        send(sock, reg_no, strlen(reg_no), 0);

        if (strcmp(reg_no, "bye") == 0) {
            printf("Connection closed.\n");
            break;
        }

        memset(buffer, 0, BUFFER_SIZE);

        int bytes = read(sock, buffer, BUFFER_SIZE - 1);

        if (bytes <= 0) {
            printf("Server disconnected.\n");
            break;
        }

        buffer[bytes] = '\0';
        printf("Server Response: %s\n", buffer);
    }

    close(sock);
    return 0;
}
```

Compilation and Execution

Terminal 1:
gcc server.c -o server
./server

Terminal 2:
gcc client.c -o client
./client

Sample Output

Connected to server.

Enter Registration Number (or type 'bye' to exit): 1002

Server Response: Name: Rahul, Branch: CSE, College: GEC_Kishanganj

Enter Registration Number (or type 'bye' to exit): 9999

Server Response: Data not found

Enter Registration Number (or type 'bye' to exit): bye

Connection closed.

Result

The TCP client-server application was implemented successfully. The client can send multiple registration-number queries over a persistent connection, and the server returns the matching student information or 'Data not found'.